24 November 2026

Struktur Data

Graf: Konsep & Implementasi

Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta



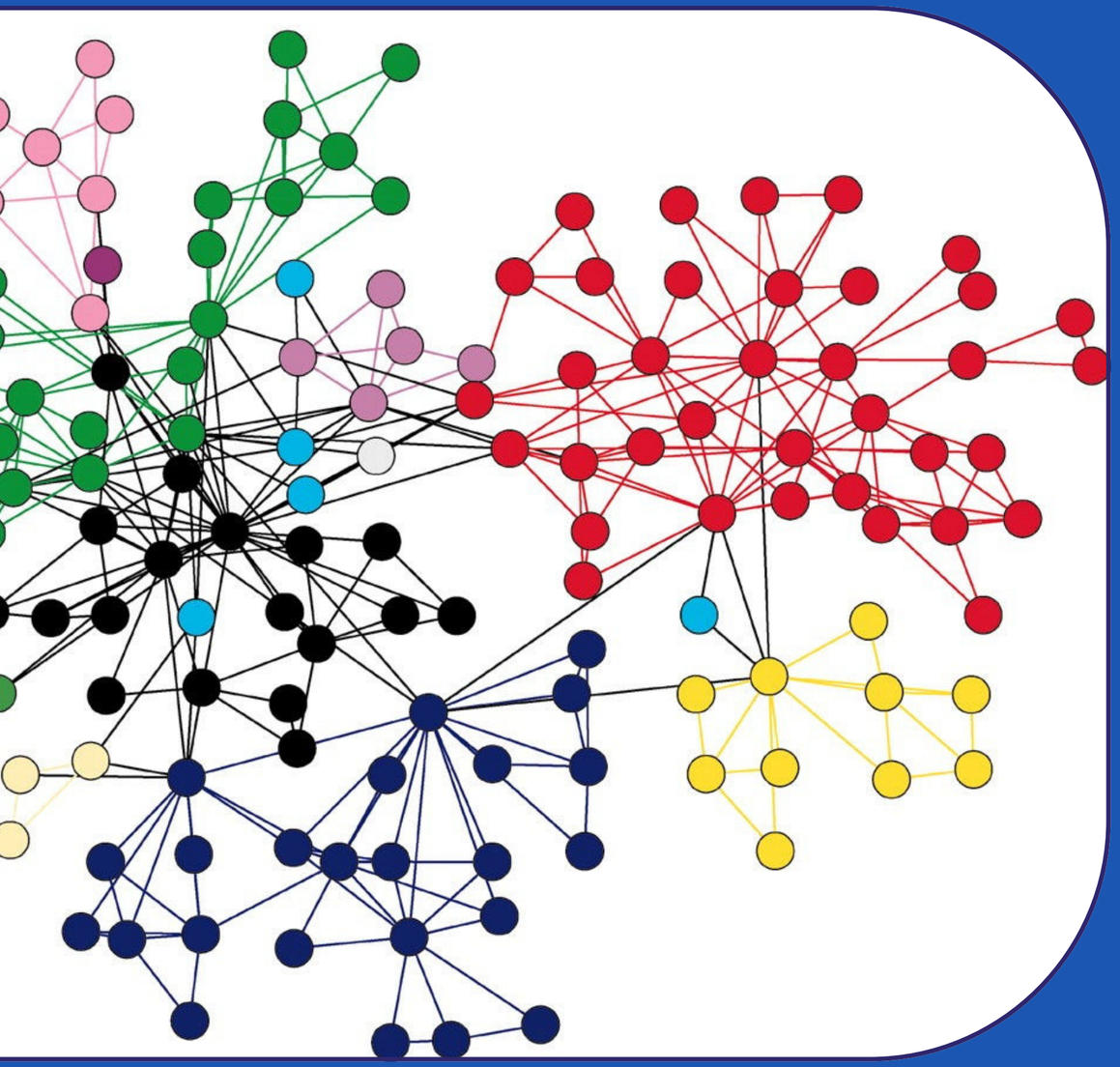
Definisi Graf

Graf adalah **struktur data** penting yang **merepresentasikan hubungan antar simpul** melalui sisi, baik terarah maupun tidak. Graf digunakan secara luas dalam berbagai bidang seperti **jaringan komputer**, **transportasi**, **jejaring sosial**, **analisis data**, **perencanaan logistik**, dan **optimasi** sistem kompleks.

Secara lebih formal, **graf** dapat didefinisikan sebagai pasangan terurut **(G = (V, E))**, dengan:

- ✓ **(V)** adalah **himpunan** yang berisi **vertex-vertex** atau **simpul-simpul** dalam graf.

- ✓ **(E)** adalah **himpunan** yang berisi **edge-edge** atau **sisi-sisi** yang menghubungkan dua vertex.



Komponen Graf

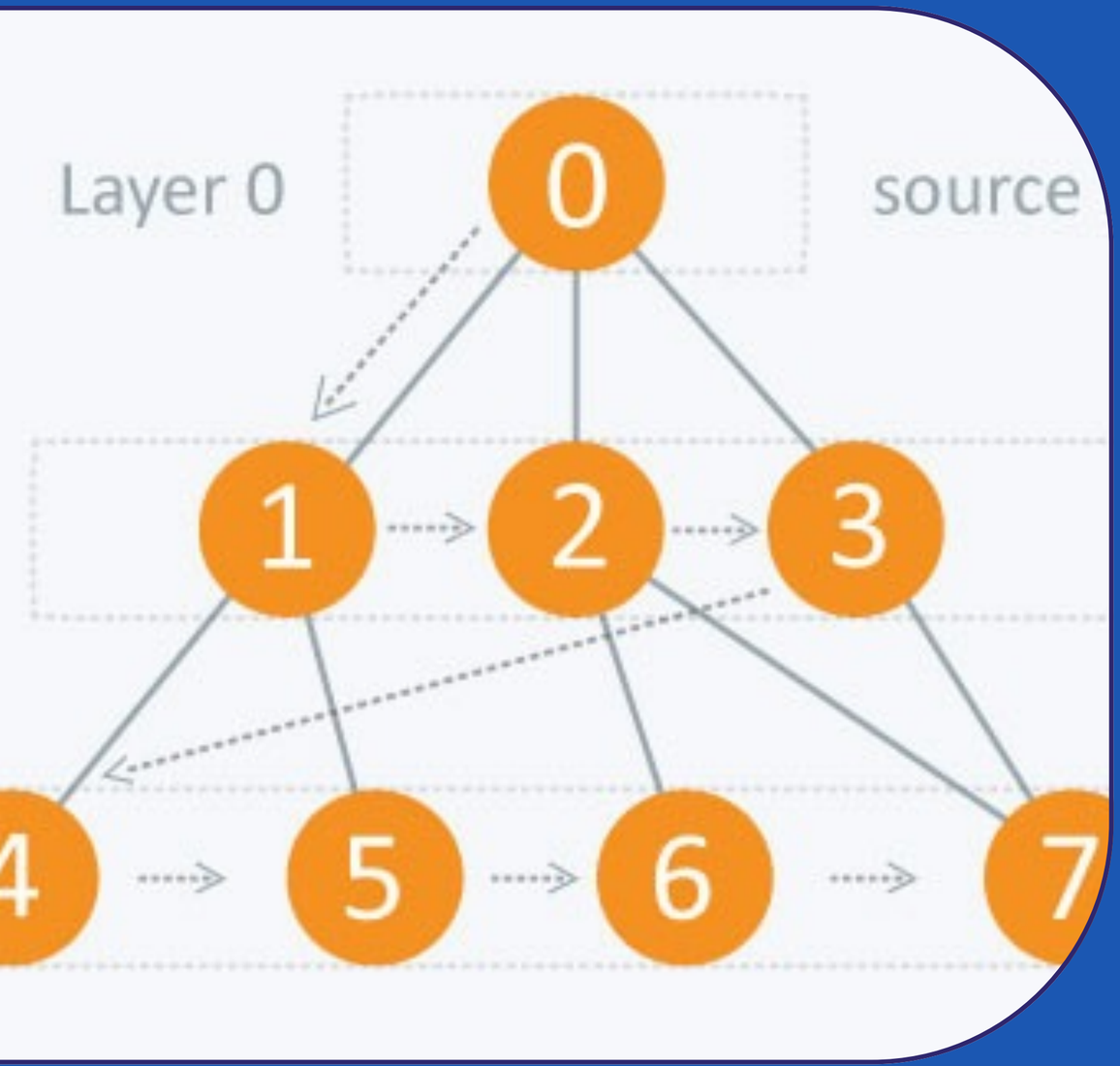
1. Vertex (Simpul)
2. Edge (Sisi)
3. Derajat (Degree)

Komponen Graf

1. **Vertex (Simpul)**: Entitas dasar graf yang mewakili objek atau titik dengan identitas unik, seperti kota dalam jaringan transportasi atau pengguna dalam jejaring sosial.

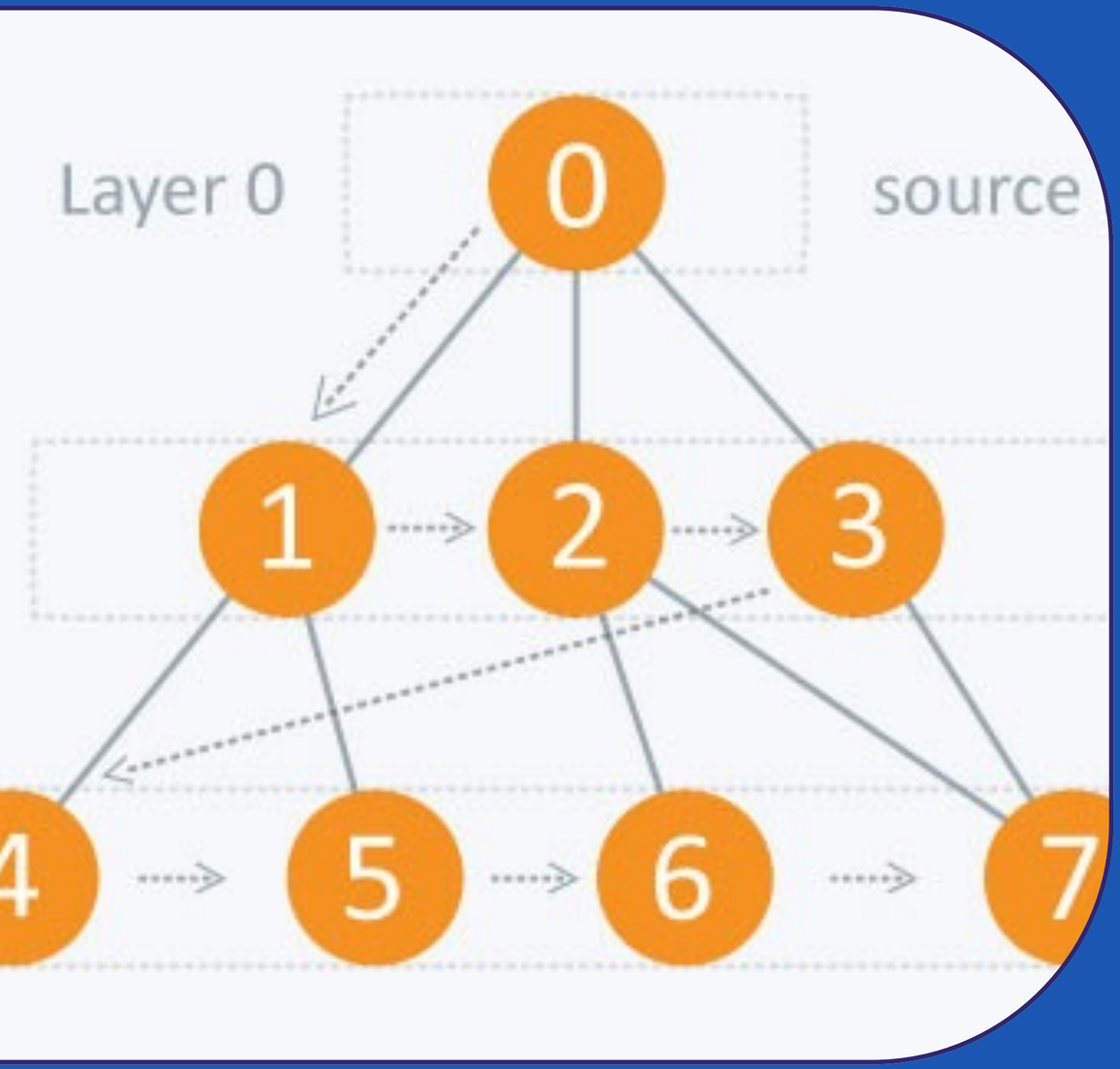
2. **Edge (Sisi)**: Hubungan antar simpul yang bisa terarah atau tidak, dan dapat memiliki bobot seperti jarak, biaya, atau waktu.

3. **Derajat (Degree)**: Jumlah sisi yang terhubung ke suatu simpul; pada graf tak berarah, derajat menunjukkan banyaknya koneksi langsung simpul tersebut. Pada graf berarah, ada dua jenis derajat: a) In-degree: Jumlah sisi yang masuk ke simpul tersebut; b) Out-degree: Jumlah sisi yang keluar dari simpul tersebut.



Jenis-Jenis Graf

1. Graf Tak Berarah (Undirected Graph)
2. Graf Berarah (Directed Graph)
3. Graf Tertimbang (Weighted Graph)
4. Graf Tidak Terhubung (Disconnected Graph)



Jenis-Jenis
Graf

5. Graf Terhubung (Connected Graph)
6. Graf Siklik (Cyclic Graph)
7. Graf Acyclic (Acyclic Graph)
8. Graf Pohon (Tree)
9. Graf Bipartite

Jenis-Jenis Graf

1. **Graf Tak Berarah (Undirected Graph)**: Sisi tidak memiliki arah, sehingga hubungan antar dua simpul bersifat timbal balik. Jika simpul A terhubung dengan simpul B, maka B juga otomatis terhubung dengan A.

2. **Graf Berarah (Directed Graph)**: Setiap sisi memiliki arah tertentu yang menunjukkan hubungan searah antar simpul. Jika terdapat sisi dari A ke B, hubungan tersebut tidak otomatis berlaku dari B ke A.

3. **Graf Tertimbang (Weighted Graph)**: Setiap sisi memiliki bobot atau nilai tertentu yang dapat menggambarkan jarak, waktu, atau biaya antar simpul. Jenis graf ini umum digunakan dalam jaringan komunikasi dan sistem transportasi.

Jenis-Jenis Graf

4. **Graf Tidak Terhubung (Disconnected Graph)**: Sebuah graf dikatakan tidak terhubung jika ada simpul yang tidak memiliki hubungan langsung dengan simpul lainnya.

5. **Graf Terhubung (Connected Graph)**: Sebuah graf dikatakan terhubung jika setiap pasang simpul dapat dihubungkan oleh sebuah jalur (path).

6. **Graf Siklik (Cyclic Graph)**: Sebuah graf dikatakan siklik jika terdapat siklus di dalamnya, yaitu jalur yang dimulai dan berakhir pada simpul yang sama.

Jenis-Jenis Graf

7. **Graf Acyclic (Acyclic Graph)**: Sebuah graf dikatakan acyclic jika tidak ada siklus di dalamnya. Salah satu contoh graf acyclic adalah pohon (tree).

8. **Graf Pohon (Tree)**: Sebuah pohon adalah graf terhubung yang tidak memiliki siklus dan terdiri dari simpul akar dan simpul cabang. Pohon memiliki sifat-sifat tertentu yang memudahkan dalam perhitungan dan analisis struktural.

9. **Graf Bipartite**: Graf bipartite adalah graf yang simpulnya dibagi dalam dua set terpisah, dan setiap sisi menghubungkan simpul dari set pertama ke simpul dari set kedua, tanpa ada sisi yang menghubungkan simpul dalam satu set yang sama.

Representasi Graf

Untuk **menyimpan** dan **mengoperasikan graf**, kita membutuhkan cara yang **efisien** dalam **menggunakan memori**. Ada **dua metode** umum yang digunakan untuk **merepresentasikan graf** dalam memori:

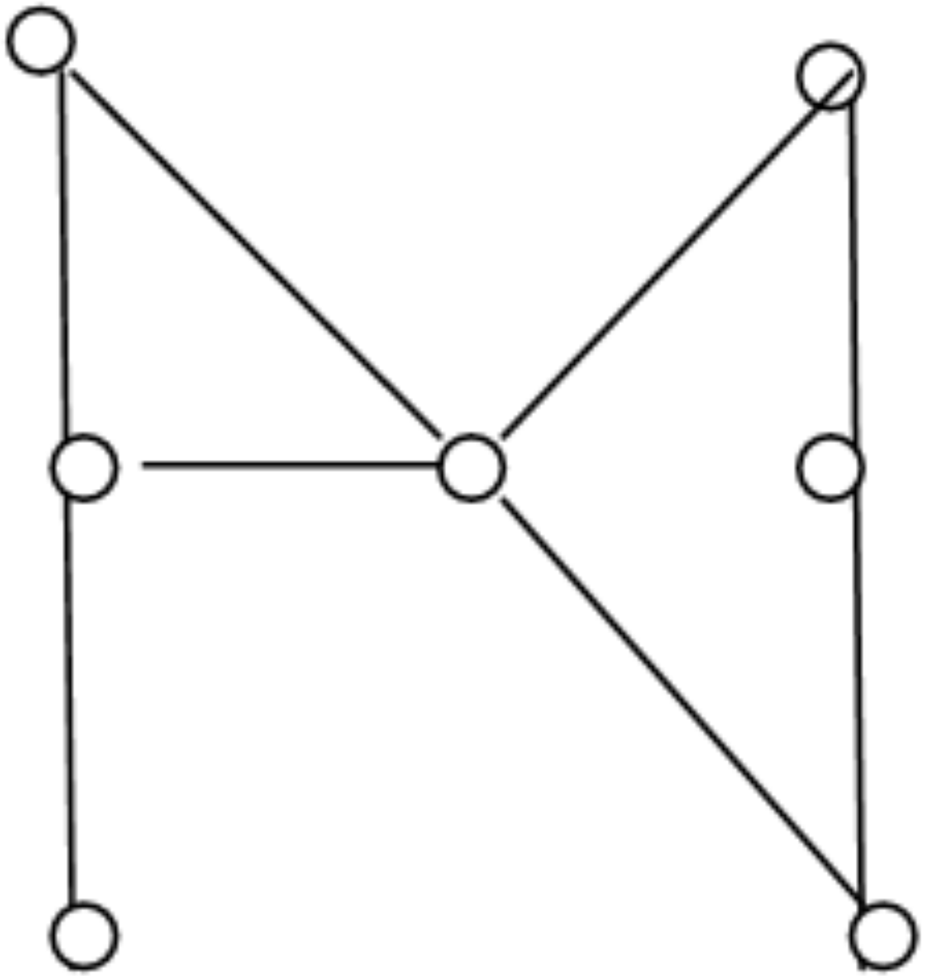
- ✓ **Adjacency matrix**: Matriks persegi dua dimensi yang menunjukkan hubungan antar simpul dalam graf. Nilai 1 atau bobot menandakan adanya sisi, sedangkan 0 tidak. **Kelebihannya** akses cepat, namun **kekurangannya** boros ruang $O(V^2)$ untuk graf jarang. Contoh adjacency matrix untuk graf berarah:

	A	B	C	D
A	0	1	0	0
B	0	0	1	0
C	1	0	0	1
D	0	0	1	0

Representasi Graf

- ✓ **Adjacency list**: Struktur data yang lebih hemat memori daripada adjacency matrix, terutama pada graf yang jarang. Pada adjacency list, setiap simpul memiliki daftar yang berisi simpul-simpul yang terhubung langsung dengannya. **Kelebihan**: Lebih hemat memori, terutama pada graf yang jarang (sparse graph). **Kekurangan**: Tidak efisien untuk memeriksa keberadaan sisi antara dua simpul, karena memerlukan pencarian dalam daftar. Contoh adjacency list untuk graf berarah:

  A -> B
  B -> C
  C -> A, D
  D -> C



Operasi Dasar
Graf

1. Menambah Vertex
2. Menambah Edge
3. Menghapus Vertex
4. Menghapus Edge
5. Mencari Jalur (Pathfinding)

Operasi Menambah Vertex

1. **Tentukan vertex baru** yang akan ditambahkan, lengkap dengan identitas uniknya.

2. **Periksa apakah vertex** sudah ada untuk mencegah duplikasi.

3. **Tambahkan** ke **struktur data graf**:
   - ✓ **Adjacency List**: buat daftar kosong untuk vertex baru.
   - ✓ **Adjacency Matrix**: tambahkan satu baris dan satu kolom baru bernilai 0.

4. **Perbarui jumlah total vertex** dalam graf.

5. **Verifikasi penambahan vertex** untuk memastikan simpul baru berhasil ditambahkan.

Operasi Menambah Edge

1. **Tentukan dua vertex** yang akan dihubungkan oleh edge baru.

2. **Periksa keberadaan vertex** untuk memastikan keduanya ada dalam graf.

3. **Tambahkan edge** sesuai **jenis graf**:
   - ✓ **Graf tak berarah**: tambahkan hubungan di kedua arah (A→B dan B→A).
   - ✓ **Graf berarah**: tambahkan hubungan hanya satu arah (misalnya A→B).

4. Jika graf berbobot, **tambahkan juga nilai bobot** (misalnya jarak, waktu, dll).

5. **Perbarui struktur data** dengan hubungan baru.

6. **Verifikasi hasil penambahan** untuk memastikan edge baru berhasil dibuat.

Operasi Menghapus Vertex

1. **Tentukan vertex** yang akan dihapus berdasarkan identitas uniknya.

2. **Periksa keberadaan vertex** untuk memastikan simpul ada dalam graf.

3. **Hapus semua edge** yang terhubung dengan vertex tersebut:
   - ✓ **Adjacency List**: hapus dari daftar tetangga.
   - ✓ **Adjacency Matrix**: hapus baris dan kolom terkait.

4. **Hapus vertex** dari struktur data graf.

5. **Perbarui jumlah vertex** dan verifikasi bahwa penghapusan berhasil.

Operasi Menghapus Edge

1. **Tentukan dua vertex** yang terhubung oleh edge yang akan dihapus.

2. **Periksa keberadaan edge** untuk memastikan hubungan tersebut ada.

3. **Hapus edge** dari **struktur data graf**:
   - ✓ **Adjacency List**: hapus entri tetangga pada kedua vertex (untuk graf tak berarah) atau satu arah (untuk graf berarah).
   - ✓ **Adjacency Matrix**: ubah nilai dari 1 atau bobot tertentu menjadi 0 atau ∞.

4. **Perbarui data graf** untuk mencerminkan perubahan koneksi.

5. **Verifikasi penghapusan edge** agar hubungan benar-benar terputus.

Operasi Mencari Jalur

1. **Tentukan simpul awal** dan **simpul tujuan** yang akan dicari jalurnya.

2. **Pilih algoritma** yang sesuai dengan **jenis graf**: a) **BFS** untuk graf tak berbobot; b) **Dijkstra** untuk graf berbobot positif; c) **A* (A-Star)** untuk pencarian dengan heuristic; d) **Floyd-Warshall** untuk semua pasangan jalur.

3. **Inisialisasi struktur data** seperti jarak awal, simpul yang sudah dikunjungi, dan antrian atau prioritas.

4. **Lakukan iterasi pencarian jalur** dengan memperbarui jarak terpendek atau nilai heuristik hingga simpul tujuan ditemukan.

5. **Rekonstruksi jalur** dari hasil pencarian untuk mendapatkan lintasan optimal.



Traversal
Graf

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)

Breadth-First Search

Breadth-First Search adalah algoritma **penelusuran graf** yang mengunjungi simpul berdasarkan **tingkat (level)**, dimulai dari simpul sumber dan meluas ke semua simpul yang terhubung secara bertahap. Menggunakan **struktur data antrian (queue)** untuk menyimpan simpul yang akan dikunjungi.

- ✓ **Langkah-langkah**: mulai dari simpul awal, masukkan ke antrian, kunjungi semua tetangga yang belum dikunjungi, tambahkan ke antrian, dan ulangi hingga seluruh simpul dikunjungi.

- ✓ **Kelebihan**: mampu menemukan jalur terpendek pada graf tak berbobot.

- ✓ **Kekurangan**: memerlukan memori besar karena harus menyimpan banyak simpul dalam antrian saat penelusuran.

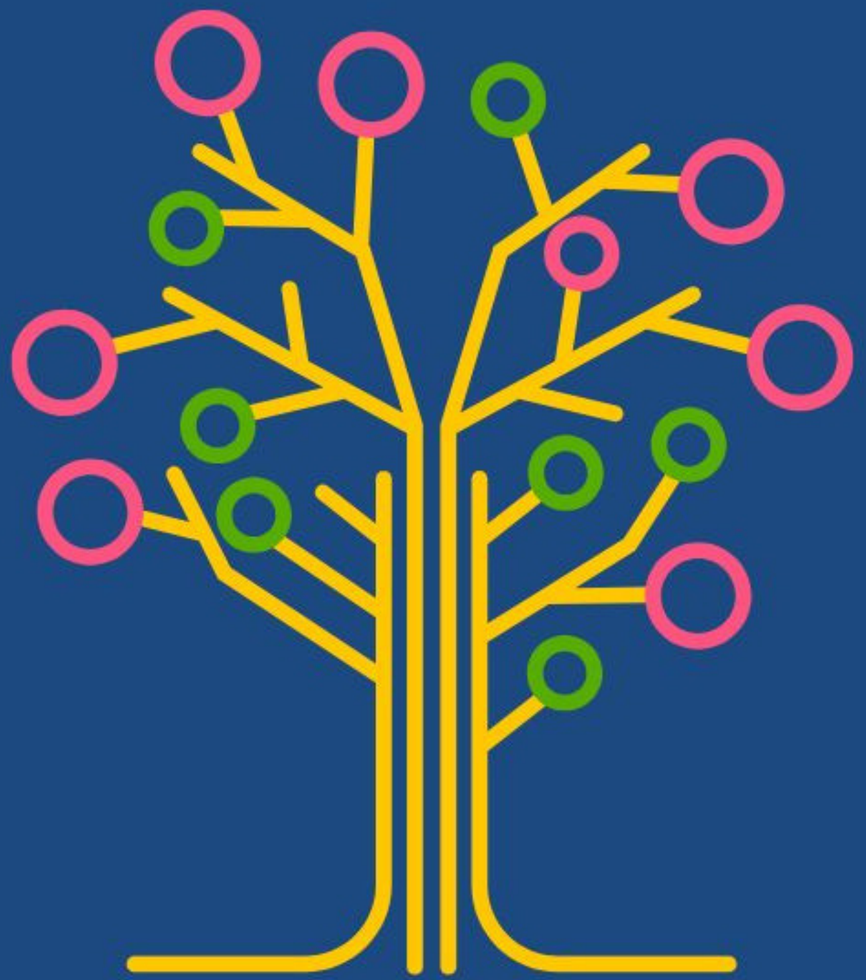
Depth-First Search

Depth-First Search adalah algoritma **penelusuran graf** yang menjelajahi **setiap cabang secara mendalam** sebelum beralih ke cabang lain. Cocok untuk masalah seperti pencarian elemen, pendeteksian siklus, dan penelusuran lintasan dalam graf. Menggunakan **rekursi atau stack** untuk melacak simpul yang sedang dikunjungi.

- ✓ **Langkah-langkah**: mulai dari simpul sumber, kunjungi simpul tetangga yang belum dikunjungi, lakukan penelusuran mendalam, dan kembali jika tidak ada tetangga tersisa.

- ✓ **Kelebihan**: membutuhkan memori lebih sedikit dibanding BFS.

- ✓ **Kekurangan**: tidak menjamin menemukan jalur terpendek dalam graf.



Algoritma
Graf

1. Dijkstra's Algorithm
2. Prim's Algorithm
3. Kruskal's Algorithm
4. Bellman-Ford Algorithm
5. Floyd-Warshall Algorithm

Algoritma Djikstra

Algoritma Djikstra digunakan untuk **mencari jalur terpendek** antara dua simpul dalam graf tertimbang tanpa bobot negatif. Bekerja dengan **prinsip greedy**, memilih simpul dengan jarak terpendek pada setiap langkah.

- ✓ **Inisialisasi**: jarak awal simpul sumber = 0, lainnya = tak hingga.
- ✓ **Proses iteratif**: perbarui jarak simpul tetangga melalui simpul terdekat, lalu pilih simpul dengan jarak minimum berikutnya. Berhenti saat semua simpul telah dikunjungi.
- ✓ **Kelebihan**: efisien dan akurat untuk graf positif (kompleksitas $O(V^2)$ atau $O(E + V \log V)$).
- ✓ **Kekurangan**: tidak dapat menangani bobot sisi negatif.

Algoritma Prim

Algoritma Prim digunakan untuk menemukan **Minimum Spanning Tree (MST)** pada graf **tertimbang** dan **terhubung**, yaitu pohon dengan total bobot sisi paling kecil. Bekerja dengan **prinsip greedy**, memilih sisi berbobot terkecil yang menghubungkan simpul yang sudah terhubung dengan simpul yang belum.

- ✓ **Langkah-langkah**: mulai dari satu simpul acak, pilih sisi terkecil yang menghubungkan ke simpul baru, tambahkan ke MST, dan ulangi hingga semua simpul terhubung.

- ✓ **Kelebihan**: efisien untuk graf terhubung dengan kompleksitas O(E log V) menggunakan heap.

- ✓ **Kekurangan**: tidak dapat digunakan pada graf yang tidak terhubung.

Algoritma Kruskal

Algoritma Kruskal digunakan untuk menemukan **Minimum Spanning Tree (MST)** dengan **memilih sisi berbobot terkecil** secara global, berbeda dari **Prim** yang berbasis simpul terdekat. Bekerja dengan **prinsip greedy**, menambahkan sisi berbobot terkecil ke MST selama tidak membentuk siklus.

- ✓ **Langkah-langkah**: urutkan semua sisi berdasarkan bobot, pilih sisi terkecil yang tidak membentuk siklus, dan ulangi hingga semua simpul terhubung.

- ✓ **Kelebihan**: sederhana, mudah diimplementasikan, dan efisien untuk graf jarang (sparse).

- ✓ **Kekurangan**: memerlukan struktur data tambahan seperti Union-Find untuk mendeteksi siklus.

Algoritma Bellman-Ford

Algoritma Bellman-Ford digunakan untuk **menemukan jalur terpendek** dalam graf tertimbang, termasuk yang memiliki **bobot sisi negatif**. Bekerja dengan **prinsip relaksasi sisi**, yaitu memperbarui jarak simpul jika ditemukan jalur yang lebih pendek. Proses relaksasi dilakukan sebanyak **(V−1)** kali, lalu memeriksa adanya siklus negatif.

- ✓ **Langkah-langkah**: inisialisasi jarak (sumber = 0, lainnya = ∞), lakukan relaksasi semua sisi (V−1 kali), lalu cek siklus negatif.

- ✓ **Kelebihan**: dapat menangani bobot negatif dan mendeteksi siklus negatif.

- ✓ **Kekurangan**: kurang efisien untuk graf besar karena kompleksitasnya O(V·E).

Algoritma Floyd-Warshall

Algoritma Floyd-Warshall digunakan untuk **mencari jalur terpendek** antara semua **pasangan simpul** dalam graf berarah maupun tak berarah. Berbeda dari Dijkstra yang fokus pada satu sumber, algoritma ini menghitung semua kombinasi jalur menggunakan pendekatan **dynamic programming**.

- ✓ **Prinsip kerja**: memperbarui jarak antar simpul dengan mempertimbangkan simpul lain sebagai perantara untuk menemukan jalur lebih pendek.

- ✓ **Langkah-langkah**: inisialisasi matriks jarak, iterasi melalui setiap simpul untuk memperbarui jarak antar pasangan simpul, hingga semua jarak optimal.

- ✓ **Kelebihan**: menghitung semua jalur terpendek sekaligus.

- ✓ **Kekurangan**: kompleksitas tinggi $O(V^3)$, kurang efisien untuk graf besar.

Aplikasi Graf

1. **Jaringan Komputer**: Graf digunakan untuk memodelkan topologi jaringan, dengan simpul sebagai perangkat dan sisi sebagai koneksi. Penerapannya mencakup routing (OSPF, RIP), analisis jalur alternatif saat gangguan, dan optimasi bandwidth.

2. **Algoritma Pencarian (Pathfinding)**: Graf membantu menemukan jalur terpendek atau optimal dalam navigasi (Google Maps, Waze) dan game AI menggunakan algoritma Dijkstra, A*, dan Bellman-Ford.

3. **Jejaring Sosial**: Simpul merepresentasikan pengguna dan sisi menunjukkan hubungan atau interaksi. Teori graf digunakan untuk analisis pengaruh (centrality), deteksi komunitas, dan pemetaan jalur informasi seperti pada Facebook Graph API dan LinkedIn.

Aplikasi Graf

4. **Sistem Rekomendasi**: Graf bipartit menghubungkan pengguna dan produk, memungkinkan metode seperti collaborative filtering dan graph-based ranking untuk memberikan rekomendasi personal (film, musik, barang) secara relevan di platform seperti Netflix, Spotify, dan Amazon.

5. **Sistem Perutean dan Logistik**: Graf digunakan untuk menentukan rute distribusi optimal menggunakan algoritma Floyd-Warshall dan Minimum Spanning Tree (Prim/Kruskal). Penerapannya mencakup pengiriman barang, layanan ride-hailing, dan navigasi robot otonom.

24 November 2026

Sekian

Terima Kasih



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta